

Clase 5

Tema 3: Selección de algoritmos para el aprendizaje automático y proceso completo de entrenamiento, optimización y despliegue del modelo

Parte 2

Ciclo: Máster en IA y Big Data

Módulo: MP2: Sistemas de aprendizaje automático

Nil Redón Orriols

Contenidos de la clase de hoy

- Fases del aprendizaje automático
- Plataformas de aprendizaje automático

Fases del aprendizaje automático

Una vez tenemos listos los datos, los puntos principales del proceso de entrenamiento de un modelo de aprendizaje automático son:

- Configuración del modelo y del proceso de entrenamiento
- Entrenamiento y validación del modelo
- Ajuste de hiperparámetros
- Optimización del modelo entrenado
- Despliegue y monitorización del modelo final

Configuración del modelo y del proceso de entrenamiento

Antes de lanzar el proceso de entrenamiento, debemos elegir:

- El modelo o modelos que vamos a entrenar.
- El framework que vamos a usar (veremos ejemplos al final de la clase).
- Si es necesario, la estrategia de inicialización del modelo: aleatoria, estadística, modelo pre-entrenado o opciones avanzadas (Xavier, He, LeCun...)

Entrenamiento y validación del modelo

Es necesario elegir la métrica que va a medir cómo de bueno es el modelo en cada paso del proceso de entrenamiento.

Para problemas de clasificación binaria, definimos:

$$\text{Log-loss} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right]$$

$$H(y, p) = - \sum_{i=1}^N y_i \log(p_i)$$

Entrenamiento y validación del modelo

Hay que definir también la métrica de evaluación final del modelo, una vez elegido el threshold sobre la probabilidad.

Para problemas de clasificación binaria, definimos:

- True Positive: Predicción 1, clase real 1
- False Negative: Predicción 0, clase real 1
- False Positive: Predicción 1, clase real 0
- True Negative: Predicción 0, clase real 0

Entrenamiento y validación del modelo

Definimos entonces:

- Matriz de confusión, para visualización intuitiva de los valores de TP, TN, FN y FP.

	Predicción Positiva	Predicción Negativa
Clase Positiva	Verdaderos Positivos (TP)	Falsos Negativos (FN)
Clase Negativa	Falsos Positivos (FP)	Verdaderos Negativos (TN)

- Se puede construir de manera análoga para problemas de clasificación multi-clase

Entrenamiento y validación del modelo

Definimos entonces:

- Accuracy (Exactitud): Más alto cuando el modelo más acierta, independientemente de qué clase

$$\frac{TP + TN}{TP + TN + FP + FN}$$

- Precisión (Precision): Más alto cuando más veces el modelo acierta cuando predice clase 1

$$\frac{TP}{TP + FP}$$

Entrenamiento y validación del modelo

Definimos entonces:

- Recall (Sensibilidad): Más alto cuando menos veces el modelo falla cuando predice clase 1

$$\frac{TP}{TP + FN}$$

- F1 score: Media armónica de precisión y recall

$$2 \cdot \frac{\text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}}$$

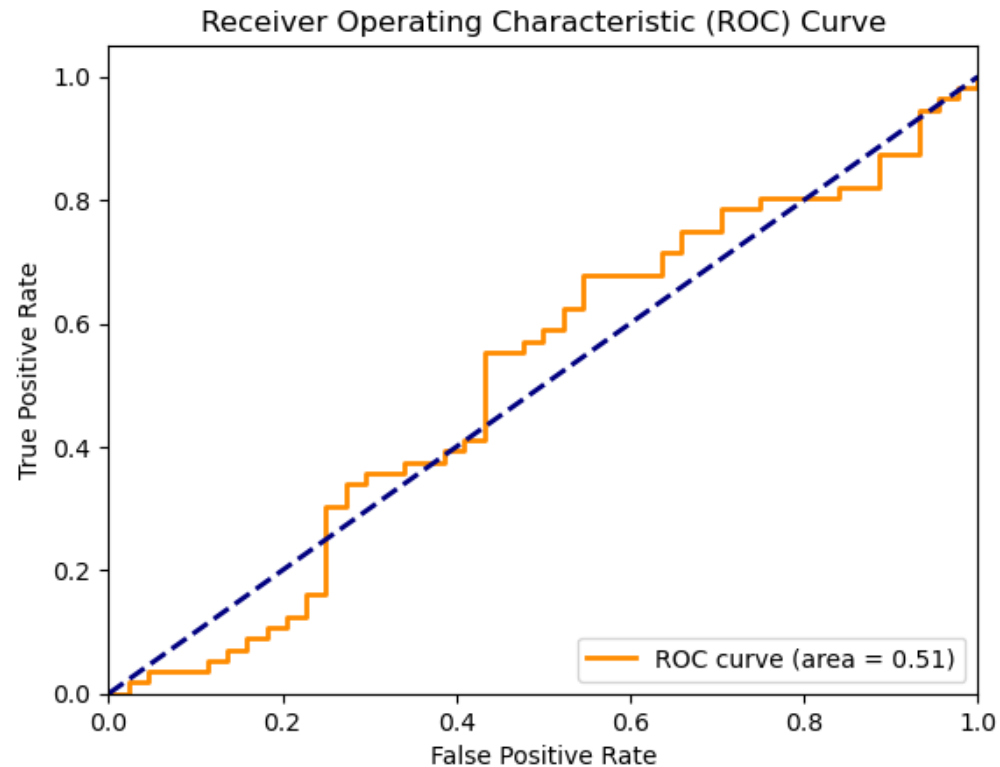
Entrenamiento y validación del modelo

Definimos entonces:

- Curva ROC (Receiver Operating Characteristic): Gráfico que se fabrica con el valor obtenido por el modelo al variar el threshold entre 0 y 1, con los FP en el eje X y los TP en el eje Y.
- AUC (Área bajo la curva ROC): Área bajo la curva. Cuanto más se acerca el valor de AUC a 1, mejor.

Entrenamiento y validación del modelo

Ejemplo aleatorio de curva ROC:



Entrenamiento y validación del modelo

- Para problemas de clasificación multiclase, se generalizan todas las definiciones y técnicas de la clasificación binaria.
- Por lo tanto, hay un valor de TP, TN, FN y FP para cada clase, y a partir de esos valores se pueden calcular todas las métricas.

Entrenamiento y validación del modelo

Para problemas de regresión, definimos primero la función de evaluación del modelo. Habitualmente, se elige una de las siguientes alternativas:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Entrenamiento y validación del modelo

Para problemas de regresión, definimos el coeficiente de determinación R^2 (entre 0 y 1) como:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Entrenamiento y validación del modelo

- Valores cercanos a 1 indican que una correlación lineal muy alta entre la variable dependiente y el modelo.
- Hay variaciones también de este coeficiente, como el R^2 ajustado, que penaliza la inclusión de variables con poca relevancia, o el pseudo- R^2 , para modelos de regresión no lineales.

Entrenamiento y validación del modelo

Pero, y cómo medimos el éxito de un modelo no supervisado?

- Para un problema de reducción de dimensionalidad, los propios modelos (como PCA) miden el porcentaje de varianza explicada en las variables de la proyección.
- Para el clustering, la mejor manera es representar gráficamente, pero hay índices que nos dan insights sobre el resultado.

Entrenamiento y validación del modelo

Para el clustering, definimos índices:

- Índice de silueta: Se calcula para cada punto y varía entre -1 y 1 , Cuanto más cercano a 1 , más “en el centro de un cluster” se encuentra un punto.
- $a(i)$ es la distancia promedio entre el punto y todos los demás puntos del mismo clúster, $b(i)$ como la distancia promedio entre el punto (i) y todos los puntos del clúster más cercano al que no pertenece

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

- Otros: Índice de Dunn (se calcula por clusters y

Entrenamiento y validación del modelo

Para el clustering, definimos índices:

- Índice de silueta: Se calcula para cada punto y varía entre -1 y 1 , Cuanto más cercano a 1 , más “en el centro de un cluster” se encuentra un punto.

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

- $a(i)$ es la distancia promedio entre el punto y todos los demás puntos del mismo clúster, $b(i)$ como la distancia promedio entre el punto (i) y todos los puntos del clúster más cercano al que no pertenece.

Entrenamiento y validación del modelo

Para el clustering, definimos índices:

- Índice de Dunn: Se calcula por clusters, y valores altos indican clusters compactos y bien separados entre ellos.
- Otros: Davies-Bouldin, Calinski-Harabasz, Rand Ajustado (ARI)

Entrenamiento y validación del modelo

- Una vez elegida la métrica, hay que elegir otros parámetros como el optimizador, el número de iteraciones máximo y otros que dependen del modelo.
- En general, se recomienda revisar y ajustar los valores default de los frameworks.
- Vamos a ver estrategias y parámetros clave para evitar el overfitting

Entrenamiento y validación del modelo

- **Regularización: L1 (Lasso), L2 (Ridge) y Elastic Net:**

La regularización es una técnica que consiste en añadir un término de penalización a la función de pérdida para evitar que los pesos se vuelvan demasiado grandes.

- Ejemplo: Fórmula para Elastic Net

$$\lambda \left(\frac{1 - \alpha}{2} \sum_{j=1}^m \hat{\beta}_j^2 + \alpha \sum_{j=1}^m |\hat{\beta}_j| \right)$$

Entrenamiento y validación del modelo

- **Dropout:** La estrategia más habitual para redes neuronales, la veremos en el Tema 4
- **Early Stopping:** Se marca una métrica objetivo a monitorizar, un umbral de mejora y un número máximo de iteraciones en que, si la función no mejora por encima del umbral, se para el entrenamiento y se restauran los pesos a la mejor iteración. Es el criterio de parada más utilizado.

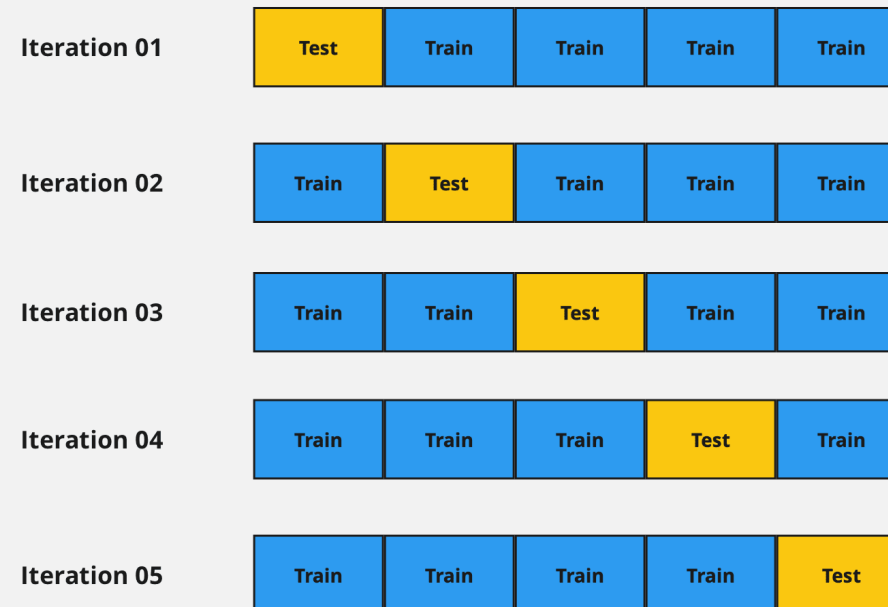
Entrenamiento y validación del modelo

- **Cross validation:** Consiste en dividir el dataset entero en Folds del mismo tamaño y a cada iteración elegir aleatoriamente un fold como validation set, y el resto como training set.
- Hay diferentes variantes para elegir de manera inteligente los Folds, como el Stratified K-fold Cross-validation

Entrenamiento y validación del modelo

- Ejemplo: Source: Dataaspirant.com

K-Fold Cross Validation



Entrenamiento y validación del modelo

- Si seguimos teniendo overfitting, quizá es necesario reducir la complejidad del modelo utilizado, o aumentar el conjunto de datos.
- También se puede dar el fenómeno contrario (underfitting), en ese caso debemos intentar utilizar modelos más complejos y reducir el uso de estrategias complementarias.

Ajuste de hiperparámetros

- Para encontrar el mejor modelo, es habitual entrenar diferentes variantes del mejor modelo candidato para obtener el mejor resultado posible.
- Hay estrategias para variar los elementos clave de la arquitectura del modelo (hiperparámetros) y encontrar la mejor combinación. Esto se conoce como ajuste de hiperparámetros.

Ajuste de hiperparámetros

Técnicas más habituales:

- Grid search: Prueba todas las combinaciones posibles en un conjunto de valores definido para cada hiperparámetro.
- Random search: Análogo al Grid search, pero se selecciona y ejecuta un número fijo de combinaciones aleatorias de hiperparámetros.

Ajuste de hiperparámetros

Técnicas más habituales:

- Bayesian optimization: Se construye un modelo probabilístico para encontrar las que posiblemente serán las mejores combinaciones de hiperparámetros, y se ejecutan solo esas combinaciones para encontrar la mejor.

Optimización del modelo entrenado

Una vez entrenado el modelo final, hay que optimizarlo (validando que no se pierde precisión) antes de desplegarlo. Técnicas más habituales:

- Cuantización: Reducir la precisión de los números utilizados en el modelo.
- Poda (pruning) de redes neuronales: Eliminar conexiones y/o neuronas con pesos bajos o bajo impacto.

Optimización del modelo entrenado

Técnicas más habituales:

- Compresión y conversión a formatos optimizados:
 - Codificación Huffman
 - Pickle
 - Joblib
 - HDF5 (específico de Keras y Tensorflow)
 - Formato ONNX (Open Neural Network Exchange) para redes neuronales
- Adaptación del modelo para optimizar su ejecución (paralelización, distribución de tareas...) en hardware específico, como TPU o GPUs.

Despliegue y monitorización del modelo final

- Ya optimizado, debemos disponibilizar el modelo en producción para su uso sobre nuevos datos.
- Lo más habitual es desplegarlo en un servidor (Cloud) accesible y securizado vía API, siguiendo las best practices de Dev-ops.
- Existe una rama llamada MLOps para los desarrolladores de modelos de aprendizaje automático.

Despliegue y monitorización del modelo final

Best practices y herramientas del MLOps:

- Control de versiones: Vital para reproducibilidad, trazabilidad y colaboración: Git, DVC, MLflow, TensorBoard.
- CI/CD: Análogo al Dev-ops: Kubernetes, Docker, Jenkins, Github, Airflow, Gitlab...
- Monitorización: Disponibilidad y precisión del modelo: Kibana, Grafana, Prometheus...

Plataformas de aprendizaje automático

Principales frameworks open-source:

- **Scikit-learn (Sk-learn)**: La librería más conocida para modelos de aprendizaje automático. Contiene clases para la gran mayoría de modelos de IA, fácilmente integrables con Numpy, Pandas, Scipy, Matplotlib, Seaborn...

Plataformas de aprendizaje automático

Principales frameworks open-source:

- **TensorFlow:** Principal framework de trabajo para redes neuronales, con herramientas para controlar todo el flujo de construcción de una red neuronal (por ejemplo, con herramientas como TensorBoard para MLOps).

Plataformas de aprendizaje automático

Principales frameworks open-source:

- **Keras y PyTorch:** Alternativas específicas para construir modelos de redes neuronales, más “sencillas”, intuitivas y flexibles que Tensorflow. Son librerías análogas y su uso depende del gusto del programador.

Cierre

- Entrega de las actividades del Tema 3 - 31/03
- Siguiendo clase: 08/04 - Tema 4, parte 1
- Si tenéis consultas, me podéis escribir por privado
- ¿Dudas/preguntas?

